

RELATÓRIO TÉCNICO: ANÁLISE DE ABORDAGEM DE ENGENHARIA DE REQUISITOS

Curso: Engenharia de Software

Estudante: Pablo Tiago Machado Vaz Dias

Atividade: SRS Tradicional x Backlog Ágil + DoR e DoD

1. CENÁRIO DE SOFTWARE ESCOLHIDO

Sistema SmartBairro: Aplicação web de gestão de estoque, controle de vendas a prazo e alertas de validade direcionada a microcomércios e mercearias locais.

2. CARACTERÍSTICAS DO SISTEMA

O sistema proposto possui especificidades dinâmicas que moldam o seu ciclo de vida de desenvolvimento:

- **Público-Alvo de Baixa Maturidade Digital:** Os usuários finais são pequenos comerciantes acostumados a controles manuais (cadernos), demandando uma interface extremamente simples, intuitiva e de fácil adaptação.
- **Escopo Evolutivo e Iterativo:** O sistema precisa ser validado e modificado rapidamente com base no feedback real dos comerciantes à medida que utilizam as primeiras versões em seus balcões.
- **Foco no Time-to-Market:** Necessidade de colocar um Mínimo Produto Viável (MVP) em operação o quanto antes para mitigar os riscos de mercado do microcomércio.

3. ABORDAGEM DE ENGENHARIA DE REQUISITOS ESCOLHIDA E JUSTIFICATIVA

Abordagem Escolhida: Backlog Ágil

Justificativa Técnica: A escolha pela abordagem de Backlog Ágil justifica-se pelo ambiente de alta incerteza e pela necessidade de entregas incrementais rápidas (iterações).

A utilização de um documento de Especificação de Requisitos de Software (SRS) Tradicional tornaria o processo engessado, gerando uma documentação extensa e estática que dificilmente acompanharia as mudanças exigidas pelo pequeno comerciante durante o uso diário.

Com o Backlog Ágil, os requisitos são mapeados em formato de Histórias de Usuário (*User Stories*), permitindo flexibilidade na priorização. Isso garante que o esforço de engenharia seja focado no que gera valor imediato para o negócio (como o alerta de vencimento de produtos), adaptando o escopo dinamicamente sem gerar refugo de documentação burocrática.

4. ESPECIFICAÇÃO DE REQUISITOS (VERSÃO REVISADA)

Nota de Revisão de Engenharia: Os requisitos abaixo foram revisados e refinados para eliminar completamente ambiguidades (termos como "rápido" ou "fácil"), subjetividades e inconsistências, aplicando métricas exatas de software.

4.1 Requisitos Funcionais (RF)

- **RF01 – Cadastro Compulsório de Itens Perecíveis:** O sistema deve fornecer uma interface para inclusão de novos produtos no estoque, exigindo obrigatoriamente o preenchimento dos campos: código de identificação, nome do produto, quantidade em estoque (unidades), preço de custo (R\$), preço de venda (R\$) e data de validade (DD/MM/AAAA).
- **RF02 – Disparo Automático de Alerta Visuais de Validade:** O sistema deve rastrear diariamente o banco de dados e exibir um alerta visual destacado em cor vermelha na tela principal de vendas para qualquer produto cujo intervalo entre a data atual e a data de validade cadastrada seja menor ou igual a 5 dias corridos.

4.2 Requisitos Não Funcionais (RNF)

- **RNF01 – Usabilidade (Eficiência de Operação):** O fluxo de fechamento de uma venda a prazo deve ser projetado de forma que o operador

consiga concluir a transação executando, no máximo, 3 cliques periféricos a partir do dashboard principal.

- **RNF02 – Desempenho (Tempo de Resposta em Larga Escala):** O tempo de resposta para a consulta de um item por meio da leitura do código de barras não deve ultrapassar o limite máximo de 2,0 segundos sob uma carga de até 100 requisições simultâneas no banco de dados.

5. DEFINITION OF READY (DoR) — CRITÉRIOS DE PRONTIDÃO

Para que qualquer História de Usuário (requisito) saia do Backlog e seja autorizada a entrar na fase de desenvolvimento, ela deve cumprir, obrigatoriamente, os 4 critérios abaixo:

1. **Padronização da Escrita:** A história deve estar descrita sob a estrutura formal: *"Como [tipo de usuário], eu quero [funcionalidade] para [valor de negócio/benefício]"*.
2. **Critérios de Aceitação Unívocos:** A história deve conter cenários de teste de aceitação claros e mensuráveis (Ex: dado que, quando, então), que sirvam de guia para a validação.
3. **Mapeamento de Dependências:** Todas as dependências técnicas, de banco de dados ou de infraestrutura necessárias para a execução da tarefa devem estar resolvidas e mapeadas.
4. **Estimativa de Esforço Concluída:** A tarefa deve ter seu esforço estimado pelo desenvolvedor responsável, garantindo que o escopo seja perfeitamente executável dentro do período de uma única iteração (Sprint).

6. DEFINITION OF DONE (DoD) — CRITÉRIOS DE CONCLUSÃO

Para que uma funcionalidade em desenvolvimento seja formalmente considerada "Concluída" (Pronta para Entrega), ela deve cumprir rigorosamente os 4 critérios abaixo:

1. **Revisão de Código por Pares (Code Review):** O código-fonte gerado deve passar pela inspeção de um par técnico, garantindo a aderência aos

padrões de arquitetura e a integração limpa (sem conflitos) na branch principal do repositório.

2. **Cobertura de Testes de Funcionamento:** A funcionalidade deve passar com 100% de sucesso por testes manuais de caminho feliz e testes de comportamento da regra de negócio, sem apresentar nenhuma exceção de sistema.
3. **Homologação em Ambiente Simulado:** A entrega deve ser implantada no ambiente de homologação (Staging/Testes), comprovando estabilidade e operabilidade idêntica ao ambiente produtivo.
4. **Atualização Documental da Funcionalidade:** Toda modificação estrutural gerada (tabelas no banco de dados, rotas de API criadas ou dicionários de variáveis) deve ser devidamente documentada no repositório técnico do projeto.